# Cours
# Calculs approchés d'intégrales

MAZZA AZZOUZ

MPSI 2-3
**mazza8azzouz@gmail.com**

9 avril 2026



# Plan

1. Introduction
2. Calculs approchés d'intégrales
   - Méthode des rectangles
   - Méthode des trapèzes
   - Méthode de Simpson
3. Utilisation de la fonction quad du module scipy.integrate

# Introduction

## Problème

- On considère les deux integrales suivantes :
  - I1 : $\int_0^6 x^2 dx$
  - I2 : $\int_0^6 xe^{-2x} + x^3 dx$
- Pour la valeur de I1, on peut calculer facilement la primitive de $f(x) = x^2$.
$$\int_0^6 x^2 dx = [\frac{1}{3}x^3] = 72$$
$\Rightarrow$Mais, pour la majorité des fonctions, la primitive de f(x) ne peut pas être calculée de façon simple, ou que cela demande des calculs trop longs.

- **Solution :** Afin de résoudre ce problème, nous utilisons quelques méthodes numérique pour calculer une valeur approchée de $\int_a^b f(x)dx$ où $f : [a : b] \to \mathbb{R}$ est une fonction continue. Ces méthodes numériques consistent à subdiviser régulièrement l'intervalle d'intégration pour approcher le calcul intégral par un calcul d'aires de figures géométriques plus simples.
- Les algorithmes étudiés pour estimer une valeur approchée d'une intégrale sont :
  1. Méthode des rectangles.
  2. Méthode des trapèzes.
  3. Méthode de simpson.

# Subdivision régulière d'un segment

## Définition

Soit [a ; b] un intervalle non vide et non réduit à un point (i.e. a<b). Soit $n \in \mathbb{N}^*$. Une subdivision régulière de [a ; b] en n + 1 points est une suite finie de n +1 réels : $(a_k)_{0\leq k\leq n}$ vérifiant :

- $a_0 = a$, $a_n = b$
- La suite de terme général $u_{k+1} - u_k$ est stationnaire

Autrement dit : pour tout $k \in [[0; n]]$ on a $a_k = a + k.\frac{b-a}{n}$

- **Sous python**

```
import numpy as np
X = np.linspace(a,b,n+1)
```

- **Exemple**

```
X = np.linspace(0,1,10+1) #n=10 et pas=0.1
array([ 0. ,  0.1,  0.2,  0.3,  0.4,  0.5,  0.6,  0.7,  0.8,  0.9,  1. ])
```

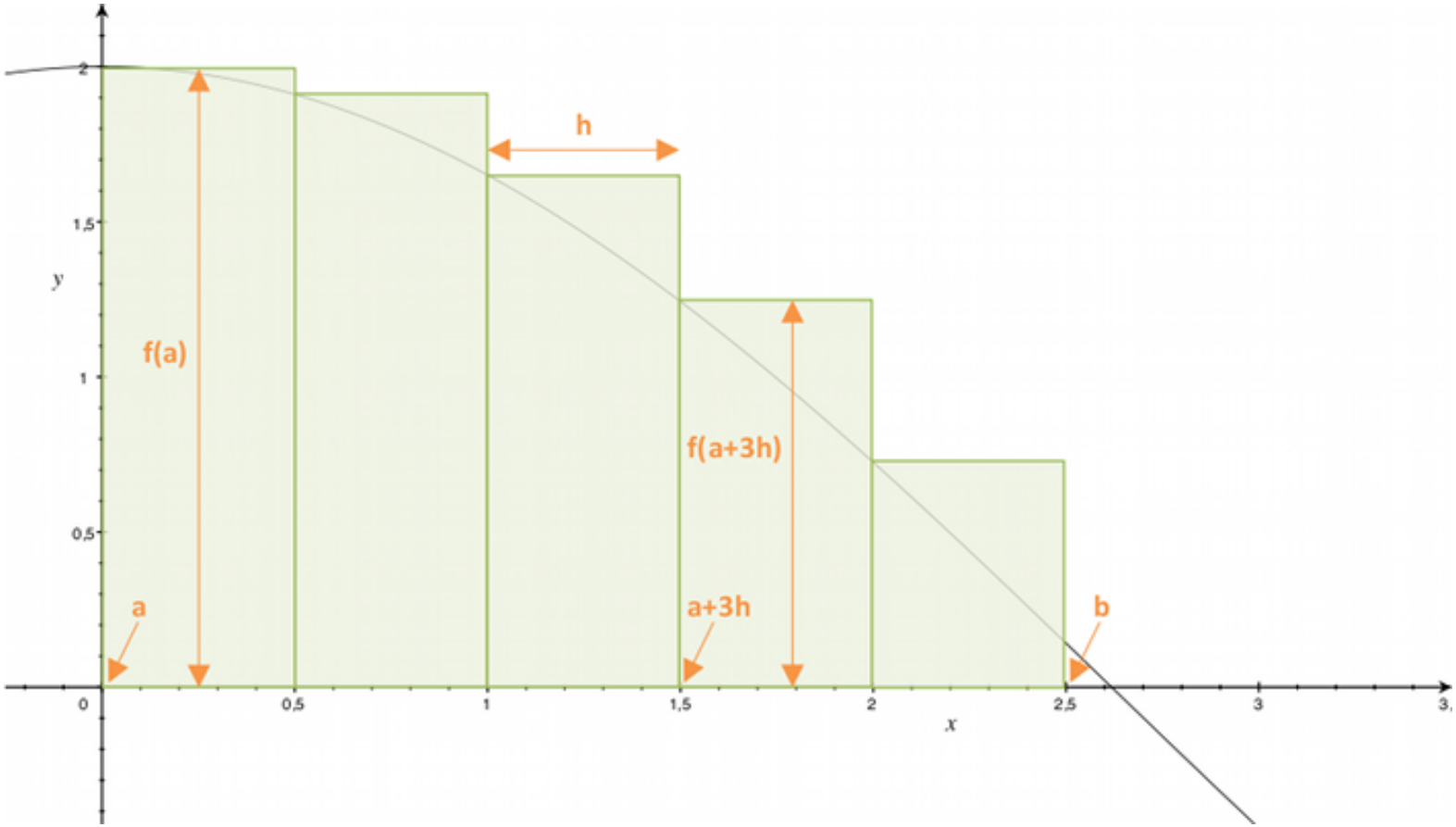
# Méthode des rectangles

## Principe

- On s'intéresse au calcul approché d'une intégrale $\int_a^b f(x)dx$ , où a, b deux nombres réels.
- La méthode des rectangles approche cette intégrale (vue comme une aire) par l'aire de $n$ rectangles de de largeur $(b-a)/n$, via la formule

$$Rec_n(f) = \frac{b-a}{n}\sum_{k=0}^{n-1} f\left(a + k\frac{b-a}{n}\right)$$

# Illustration Graphique de la méthode des rectangles



# Implémentation de la méthode des rectangles en Python

```
######Solution 1
def rectangles_gauche(f,a,b,n) :
  I=0
  pas=(b-a)/n
  for i in range(n) :
    Xi=a+i*pas
    I=I+pas*f(Xi)
  return I
```

Listing 1: la méthode des rectangles gauches

# Implémentation de la méthode des rectangles en Python

```
######Solution 2
import numpy as np
def rectangles_gauche(f,a,b,n) :
  I=0
  X=np.linspace(a,b,n)
  for i in range(n) :
    I=I+(b-a)/n *f(X[i])
  return I
```

Listing 2: la méthode des rectangles gauches

# Implémentation de la méthode des rectangles en Python

```
import matplotlib.pyplot as plt
def f(x):
    return x**2
a,b,n = 0,6,50
pas = (b - a) / n
x_rect = np.linspace(a, b - pas, n)
y_rect = f(x_rect)
# Calcul de l'approximation de l'intégrale
I_approx = rectangles_gauche(f,a,b,n)
print(f"Approximation de l'intégrale : {I_approx}")
# Tracé
x = np.linspace(a, b, 1000)
plt.plot(x, f(x), label='$f(x) = x^2$', color='blue')
# Tracer les rectangles
for i in range(n):
    plt.bar(x_rect[i], y_rect[i], width=pas, align='edge', color='orange', alph
plt.xlabel('$x$')
plt.ylabel('$f(x)$')
plt.title("Méthode des rectangles pour approximer l'intégrale")
plt.legend()
plt.grid(True)
plt.show()
```

# Illustration des rectangles gauches

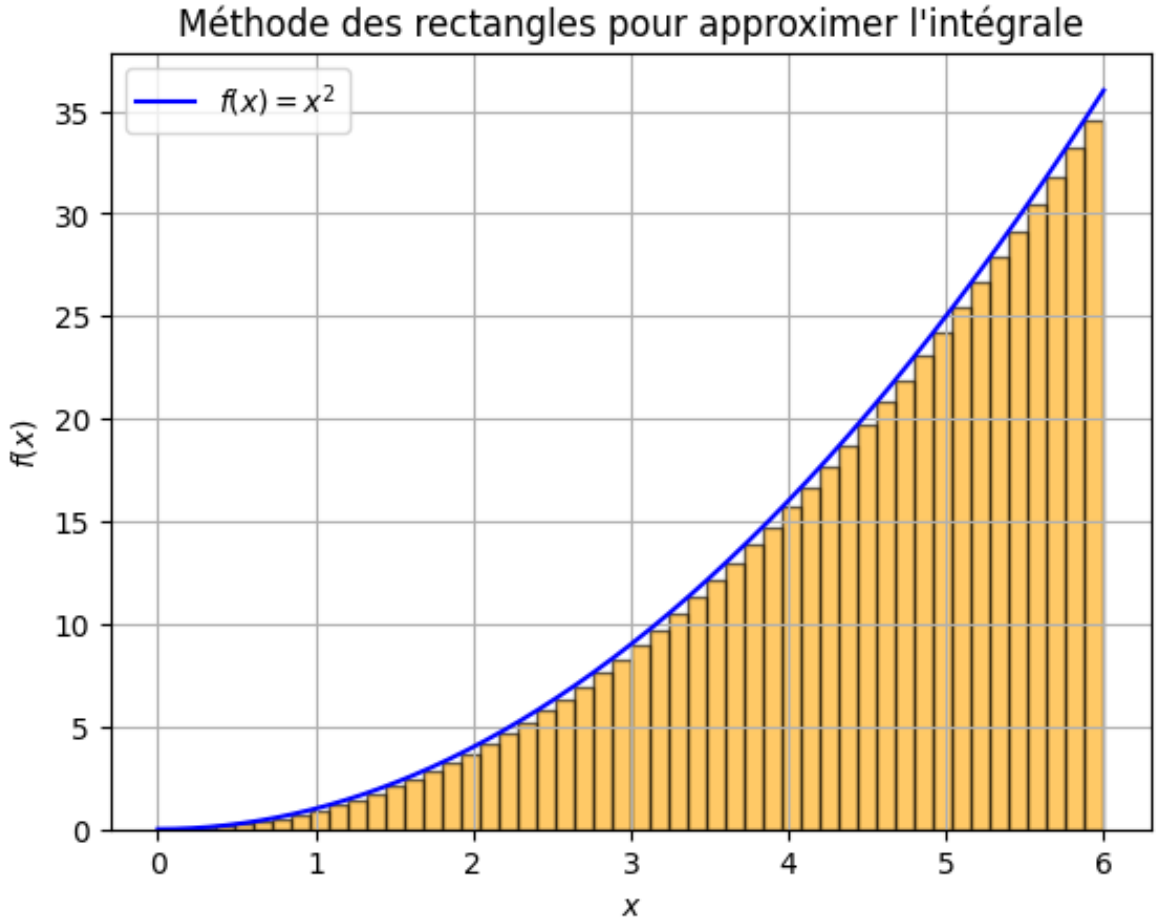



Figure – Approximation de l'intégrale définie de $f(x) = x^2$ sur $[0, 6]$ par la méthode des rectangles à gauche.

# Implémentation de la méthode des rectangles en Python

```
def RectangleMilieu(f,a,b,n) :
  I=0
  pas=(b-a)/n
  for i in range(n) :
    Xi1=a+i*pas
    Xi2=a+(i+1)*pas
    H=((Xi1+Xi2)/ 2)#le milieu de l'intervalle
    I=I+pas*f(H)
  return I
```

Listing 4: la méthode des rectangles

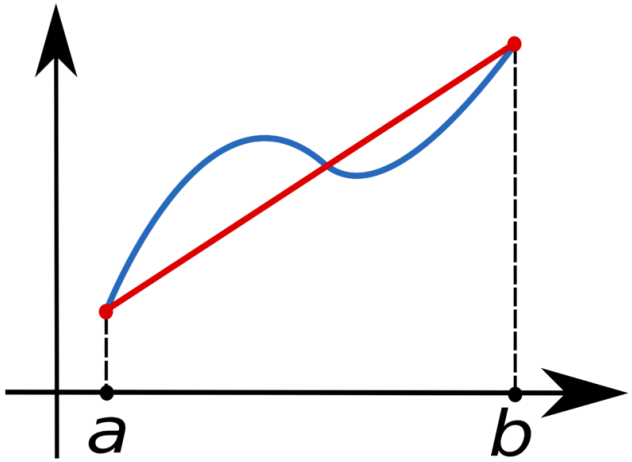
# Méthode des trapèzes

## Principe

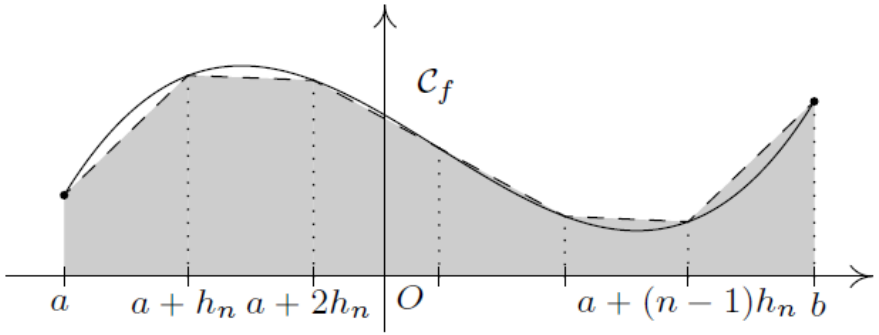
- On s'intéresse au calcul approché d'une intégrale $\int_a^b f(x)dx$ , où a, b deux nombres réels.
- La méthode des trapèzes approche cette intégrale (vue comme une aire) par l'aire de $n$ trapèzes de de largeur $(b-a)/n$, via la formule

$$Trap_n(f) = \frac{b-a}{n}\left[\frac{f(a)+f(b)}{2} + \sum_{k=0}^{n-1} f\left(a + k\frac{b-a}{n}\right)\right]$$

# Illustration Graphique de la méthode des trapèzes




$$T = (b-a)\frac{f(a)+f(b)}{2}.$$



# Implémentation de la méthode des trapèzes en Python

```
def trapezes(f,a,b,n) :
  I=0
  pas=(b-a)/n
  for i in range(n) :
    Xi1=a+i*pas
    Xi2=a+(i+1)*pas
    I=I+pas*(f(Xi1)+f(Xi2))/2
  return I
```

Listing 5: la méthode des rectangles

# Implémentation de la méthode des trapèzes en Python

```
import numpy as np
def trapezes(f,a,b,n) :
  I=0
  X=np.linspace(a,b,n)
  for i in range(n) :
    I=I+(b-a)/n *(f(X[i])+f(X[i+1]))/2
  return I
```

Listing 6: la méthode des trapèzes

# Méthode de simpson

## Principe

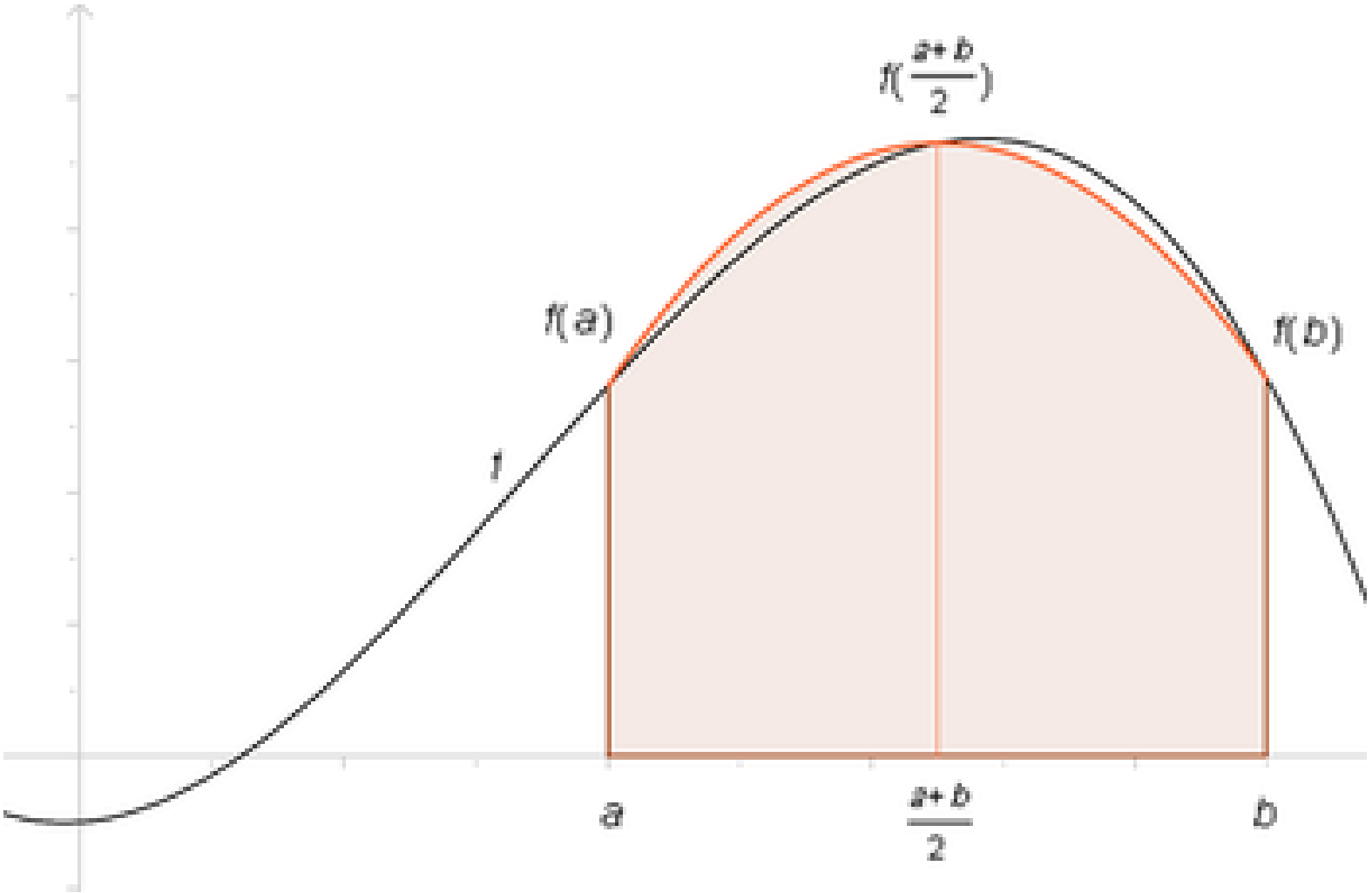
- On note toujours $a_k = a + k.\frac{b-a}{n}$
- La méthode de Simpson consiste à approximer le graphe de la fonction sur chaque segment $[a_k, a_{k+1}]$ par un arc de parabole qui coïncide avec le graphe de la fonction aux points d'abscisses $a_k$, $a_{k+1}$ et $\frac{a_k+a_{k+1}}{2}$
- En notant $f_k$ la fonction parabolique obtenue pour le segment $[a_k, a_{k+1}]$ et $m_k = \frac{a_k+a_{k+1}}{2}$, on peut montrer que l'aire sous l'arc de la parabole ainsi construite est égale à

$$\int_{a_k}^{a_{k+1}} f_k(t)dt = \frac{1}{6}.(a_{k+1} - a_k)\left[f(a_k) + 4f(m_k) + f(a_{k+1})\right]$$

- Le résultat après sommation est une expression d'une valeur approché de l'intégrale via la formule :

$$Simpson_n(f) = \int_a^b f(x)\,dx \approx \frac{h}{3}\left[f(x_0) + 4\sum_{\substack{i=1 \\ i \text{ impair}}}^{n-1} f(x_i) + 2\sum_{\substack{i=2 \\ i \text{ pair}}}^{n-2} f(x_i) + f(x_n)\right]$$

# Illustration Graphique de la méthode de Simpson



# Implémentation de la méthode de Simpson en Python

```python
import numpy as np
def simpson1(f,a,b,n):
# Vérifier que n est pair
    if n % 2 != 0:
        raise ValueError("n doit être un nombre pair pour la méthode de Simpson
    pas = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = f(x)
# Simpson : f(x0) + f(xn) + 4*impairs + 2*pairs
    I_simpson = (pas/3)*(y[0]+y[-1]+4*np.sum(y[1:-1:2])+2*np.sum(y[2:-2:2]))

    return I_simpson
```

Listing 7: la méthode de Simpson

# Implémentation de la méthode de Simpson en Python

```
######Solution 2
def simpson2(f,a,b,n) :
  S=0
  pas=(b-a)/n
  for i in range (0,n) :
    x1=a+i*pas
    x2=a+(i+1)*pas
    S=S+(f(x1)+4*f((x1+x2)/2)+f(x2))*(x2-x1)/6
  return S
```

Listing 8: la méthode de Simpson

# La fonction quad

## Défintion

- La fonction quad du module scipy.integrate permet de calculer une valeur approchée l'intégrale
- La fonction quad trouve l'intégrale d'une fonction d'une variable entre deux points.

## Syntaxe et utilisation

```
>>>from scipy.integrate import quad
>>>s=quad(func, a, b)
```

1. func : Fonction à intégrer
2. a : borne inférieur de l'intégrale
3. b : borne supérieur de l'intégrale

La fonction quad retourne un tuple s contenant deux éléments s[0] et s[1]
s[0] valeur approchée l'intégrale
s[1] : une estimation de l'erreur
Pour une syntaxe plus complète, faire help(quad)

# Exemples d'uilisation de la fonction quad

- Exemple1 :

```
from scipy.integrate import quad
f=lambda x: x **2
s, err= quad (f ,0 ,6)
print(s)
572.00000000000001# val. exacte 72
print(err)
77.993605777301129 e -13
```

- Exemple2 :

```
import numpy
f=lambda x: sin(x)
I=quad (f ,0 ,numpy.pi)
print(I)
(2.0, 2.2204460492503131e-14)
```